Informe de Laboratorio 04

Título de la práctica: MatrLab: aplicación multiplataforma en Flutter para la matrícula en laboratorios

Nota

Integrante(s)	Escuela	Asignatura
Hilacondo Begazo, Andre Jimmy ahilacondo@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Plataformas Emergentes

Número de práctica	Título de la práctica	Docente(s)
04	MatrLab: aplicación multiplataforma en Flutter para la matrícula en laboratorios	MSc. Norman Patrick Harvey Arce

Año lectivo	Nro. Semestre	Fecha de presentación	Hora de presentación
2026	B	24/10/2026	

Índice

1. Introducción **6**
1.1. Propósito del laboratorio . . . 6
1.2. Trabajo realizado y su ubicación en el informe . . . 6
1.3. Alcance y límites . . . 7

2. Investigación: frameworks multiplataforma y componentes de layout **7**
2.1. Qué es un framework multiplataforma . . . 7
2.2. Comparación de los tres frameworks . . . 7
2.3. Elección del framework . . . 8
2.4. El modelo de layout de Flutter . . . 9
2.5. Catálogo de widgets de layout usados en MatrLab . . . 10
2.6. Equivalencias con React Native y .NET MAUI . . . 11
2.7. Diseño adaptable . . . 12
2.8. Componentes de Material usados . . . 12
2.9. Errores típicos de layout y cómo se evitaron . . . 13

3. Descripción general del programa **14**
3.1. Contexto y problema . . . 14
3.2. Árbol de problemas . . . 14
3.3. De las causas a las funcionalidades . . . 14
3.4. Requisitos no funcionales . . . 16
3.5. Datos de ejemplo . . . 16

4. Diseño de interfaces **16**
4.1. Sistema de diseño . . . 16
4.2. Mapa de navegación . . . 17
4.3. Mockups . . . 17

5. Descripción del proceso de desarrollo **27**
5.1. Entorno y creación del proyecto . . . 27
5.2. Estructura del proyecto y arquitectura . . . 28
5.3. Paso 1: el estado y las reglas de negocio . . . 28
5.4. Paso 2: widgets reutilizables . . . 29
5.5. Paso 3: las pantallas . . . 30
5.5.1. Inicio de sesión . . . 31

5.5.2. Estructura principal y lista de cursos 31
5.5.3. Detalle de curso y de grupo 33
5.5.4. Confirmación y horario 36
5.5.5. Casos especiales: cruce, grupo lleno y lista de espera 37
5.5.6. Panel del administrador 38
5.5.7. Diseño adaptable 39
5.6. Del mockup a la aplicación 39
5.7. Pruebas 41
5.8. Registro de cambios 42

6. Apreciación de la herramienta **42**
6.1. Dos características a favor 42
6.2. Dos características en contra 43

7. Conclusiones **43**

A. Cómo reproducir el proyecto y las capturas **45**

Índice de figuras

1. Contribución de cada criterio al puntaje ponderado. Flutter obtiene 4.70, React Native 4.00 y .NET MAUI 3.25. . . . 9
2. Protocolo de layout de Flutter: el padre pasa restricciones (mínimo y máximo de ancho y alto) a cada hijo, cada hijo devuelve un tamaño dentro de esos límites y el padre decide dónde colocarlo. Elaboración propia a partir de [5]. . . . 9
3. Árbol de problemas de MatrLab: seis causas (abajo), problema central (centro) y cuatro efectos (arriba). . . . 14
4. Trazabilidad causa → funcionalidad → pantalla. . . . 15
5. Mapa de navegación de MatrLab. . . . 17
6. Mockup: Pantalla de inicio de sesión. . . . 18
7. Mockup: Pestaña Cursos (inicio). . . . 19
8. Mockup: Detalle de un curso. . . . 20
9. Mockup: Detalle de un grupo. . . . 21
10. Mockup: Hoja de confirmación de matrícula. . . . 22
11. Mockup: Matrícula confirmada. . . . 23
12. Mockup: Pestaña Mi horario. . . . 24
13. Mockup: Pestaña Alertas. . . . 25
14. Mockup: Pestaña Panel (administrador). . . . 26
15. Mockup: Versión para tableta (1000 × 640 dp). . . . 27
16. Arquitectura: las pantallas leen el estado a través de `MatrLabScope`; cuando el estado cambia llama a `notifyListeners()` y se reconstruyen las pantallas que dependen de él. . . . 28
17. Inicio de sesión. . . . 32
18. Lista de cursos con búsqueda. . . . 34
19. Del curso al grupo y la confirmación de la matrícula. . . . 36
20. Confirmación y horario semanal. . . . 37
21. Casos que resuelven las causas C4 y C5 del árbol de problemas. . . . 38
22. Causas C6 y C5: gestión de cupos y aviso al estudiante. . . . 39
23. Cursos en tableta: `NavigationRail` y cuadrícula de dos columnas. . . . 40
24. Horario en tableta: el contenido se limita a 720 dp y se centra. . . . 40
25. Mockup frente a implementación: login, detalle de grupo y panel. . . . 41

Índice de tablas

1. Resumen de lo desarrollado en el laboratorio y su ubicación en el informe. 6
2. Comparación de Flutter, React Native y .NET MAUI según su documentación oficial. 8
3. Matriz de decisión ponderada (valoración propia del autor). 8
4. Comportamiento de los widgets de layout más usados. 10
5. Widgets de organización y layout usados en la aplicación. 11
6. Equivalencias de layout entre los tres frameworks. 12
7. Componentes Material empleados. 13
8. Problemas comunes de layout y su prevención en MatrLab. 13
9. Requisitos funcionales derivados del árbol de problemas. 15
10. Requisitos no funcionales. 16
11. Cursos y grupos de demostración (ocupados/total). Los de Plataformas Emergentes, Seguridad Informática y Base de Datos II provienen del Laboratorio 2 [3]; * son ficticios. Los casos están elegidos a propósito: PE-A y RC-A se cruzan; BD2-B está lleno; PE-A y RC-B están casi llenos. 16
12. Sistema de diseño de MatrLab. 17
13. Registro de cambios del trabajo. 42

1. Introducción

1.1. Propósito del laboratorio

En este laboratorio desarrollé una aplicación móvil con un **framework multiplataforma** (Flutter) y armé todas sus interfaces con los **componentes propios del framework** (widgets). Las funcionalidades de cada pantalla salen del **árbol de problemas** que planteé para el caso. Además, investigué los componentes de Flutter que sirven para **organizar la interfaz y manejar el layout**, y los comparé con los de React Native y .NET MAUI para justificar la elección [1].

La aplicación es **MatrLab**, la app de matrícula en laboratorios de la Escuela Profesional de Ingeniería de Sistemas que analicé en el Laboratorio 1 (comparación de herramientas sin código) y prototipé en Adalo en el Laboratorio 2 [2, 3]. Ahora la rehago en código con Flutter, lo que me permite comparar ambos enfoques con el mismo problema.

1.2. Trabajo realizado y su ubicación en el informe

El laboratorio se organizó en cuatro frentes de trabajo: elegir el framework, investigar el layout, diseñar las interfaces a partir del árbol de problemas y programar la aplicación. La tabla 1 resume el trabajo realizado en cada uno y la sección en que se documenta.

Frente de trabajo	Trabajo realizado	Dónde
Elección del framework	Comparé Flutter, React Native y .NET MAUI con su documentación oficial y decidí con una matriz ponderada	Secc. 2
Investigación de layout	Estudié el modelo de restricciones, armé un catálogo de widgets de layout, sus equivalencias con otros frameworks y los errores típicos	Secc. 2.4 a 2.9
Funcionalidades a partir del árbol de problemas	Planteé 6 causas, de ellas 6 funcionalidades y 6 pantallas	Secc. 3
Descripción del programa	Describí el contexto, el alcance, los actores y los datos de ejemplo	Secc. 3
Diseño de interfaces	Definí el sistema de diseño y el mapa de navegación, y dibujé 10 mockups con cada elemento asociado a su widget	Secc. 4
Desarrollo de la aplicación	Estructuré el proyecto, programé el estado y las pantallas y documenté el resultado con 17 capturas comentadas	Secc. 5
Valoración del framework	Argumenté dos características a favor y dos en contra, con evidencia de este proyecto	Secc. 6
Código fuente	Se entrega completo en el repositorio del proyecto (carpetas `lib/` y `test/`)	Repositorio
Fuentes consultadas	Documentación oficial de los tres frameworks y mis informes previos	Referencias

Tabla 1: Resumen de lo desarrollado en el laboratorio y su ubicación en el informe.

1.3. Alcance y límites

- **Qué sí es:** una aplicación Flutter funcional, con 9 pantallas, lógica de matrícula (cupos, cruces de horario, lista de espera) y un panel de administración, probada con pruebas automáticas.
- **Qué no es:** no tiene servidor ni base de datos; los datos viven en memoria y se reinician al cerrar la app. No se publicó en tiendas. El control de concurrencia real (muchos estudiantes a la vez) exige un servidor y queda como trabajo futuro (secc. 7).
- **Origen de las capturas:** se generan ejecutando la aplicación real con el motor de pruebas de Flutter (`flutter_test`), que renderiza los mismos widgets que un teléfono; no son fotos de un dispositivo físico.
- **Datos:** los códigos y grupos de Plataformas Emergentes, Seguridad Informática y Base de Datos II vienen del caso del Laboratorio 2; Redes e Ingeniería de Software son ficticios, solo sirven para probar el diseño.

2. Investigación: frameworks multiplataforma y componentes de layout

2.1. Qué es un framework multiplataforma

Un framework multiplataforma permite escribir una sola base de código y ejecutarla en varias plataformas. Lo que distingue a los tres analizados es *cómo dibujan la interfaz*:

- **Flutter** no usa los controles del sistema: dibuja su propia interfaz. En plataformas nativas lo hace con el motor Impeller, que viaja dentro de la app [6].
- **React Native** describe la interfaz con React y, al ejecutarse, crea las vistas nativas de Android e iOS; no usa una vista web [11].
- **.NET MAUI** usa C# para la lógica y XAML para la interfaz, y unifica las APIs de Android, iOS, macOS y Windows en una sola, sin perder el acceso a cada plataforma [12].

2.2. Comparación de los tres frameworks

La tabla 2 resume lo que dice la documentación oficial de cada uno sobre los aspectos que importan para este laboratorio.

Aspecto	Flutter	React Native	.NET MAUI
Lenguaje	Dart [9]	JavaScript con React [11]	C# y XAML [12]
Cómo dibuja la UI	Motor propio (Impeller en nativo) [6]	Vistas nativas de Android e iOS [11]	Aplicación nativa; la capa de MAUI consume las APIs nativas [12]
Unidad de interfaz	Widget: «casi todo es un widget» [9, 6]	Componente (`View`, `Text`, `Image`...) [11]	Control y página en XAML [12]
Modelo de layout	Widgets de layout (`Row`, `Column`, `Stack`...) con restricciones [4, 5]	Flexbox (motor Yoga) mediante la propiedad `style` [10]	Clases de layout: `Grid`, `StackLayout`, `FlexLayout`, `AbsoluteLayout` [13]
Plataformas	Android, iOS, Windows, macOS, Linux y web [6]	Android e iOS (núcleo de la documentación) [11]	Android, iOS, macOS y Windows [12]
Estado de la UI	`StatefulWidget` y `setState()` [9]	Hooks como `useState` [9]	Enlace de datos (*data binding*) [12]
Navegación	`Navigator.push` con rutas [9]	Navegadores de pila y de pestañas [9]	Varios tipos de página (p. ej. *flyout*) [12]
Edición en vivo	*Hot reload* con estado conservado (VM en desarrollo) [6]	— (no verificado en las fuentes citadas)	*.NET Hot Reload* y *XAML Hot Reload* [12]
Requisito especial	Ninguno para Android	Ninguno para Android	Compilar para iOS/macOS exige una Mac [12]

Tabla 2: Comparación de Flutter, React Native y .NET MAUI según su documentación oficial.

2.3. Elección del framework

Para decidir se definieron seis criterios con un peso cada uno y se puntuó de 1 a 5. **Los pesos y puntajes son una valoración propia**, basada en lo que dice la documentación y en la necesidad concreta de MatrLab (listas, formularios, tablas, una navegación adaptable y muchas decisiones de layout); no son una medición experimental.

Criterio	Peso	Flutter	React Native	.NET MAUI
Control y claridad del layout (el foco del laboratorio)	25 %	5	4	4
Consistencia visual entre plataformas	15 %	5	4	3
Curva de aprendizaje con lo que ya se conoce	15 %	3	4	2
Herramientas para desarrollar y probar en Android	15 %	5	4	3
Documentación oficial sobre layout y componentes	15 %	5	4	4
Adecuación a MatrLab (listas, tablas, formularios, Material)	15 %	5	4	3
Puntaje ponderado	100 %	**4.70**	**4.00**	**3.25**

Tabla 3: Matriz de decisión ponderada (valoración propia del autor).

Por qué Flutter. Primero, el modelo de layout de Flutter es el más explícito: cada decisión (alinear, repartir espacio, superponer, hacer scroll) corresponde a un widget con nombre propio, lo que facilita documentar «qué hace cada componente», que es lo que pide el laboratorio. Segundo,

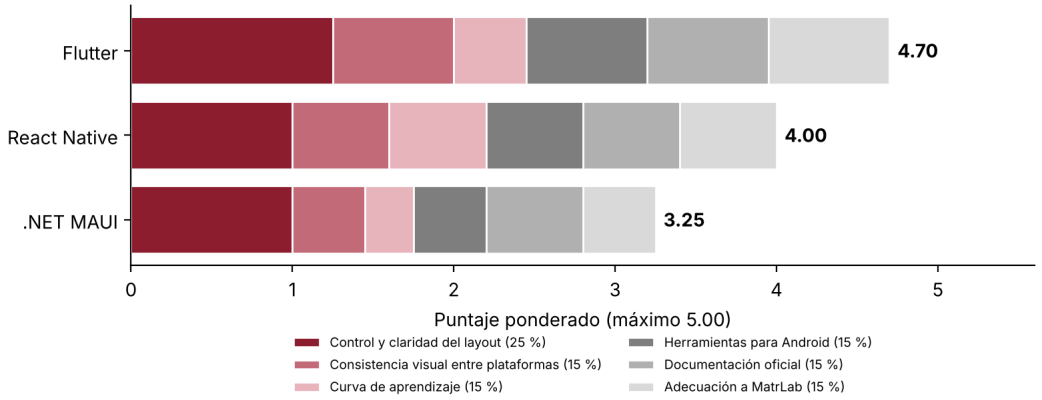


Figura 1: Contribución de cada criterio al puntaje ponderado. Flutter obtiene 4.70, React Native 4.00 y .NET MAUI 3.25.

al dibujar su propia interfaz, el mismo diseño se ve igual en todos los dispositivos, lo que simplifica las capturas y el diseño adaptable. Tercero, el entorno de Android Studio que ya se usa en el curso permite ejecutar Flutter sin herramientas nuevas.

Por qué no los otros dos. React Native es una alternativa sólida y más cercana a JavaScript (el autor ya la usó en un trabajo previo), pero su layout se expresa con propiedades de estilo y no con componentes con nombre, lo que hace menos visible el «catálogo» de componentes de organización. .NET MAUI obliga a trabajar con C# y XAML y, para iOS, a disponer de una Mac [12]; además la curva de aprendizaje es la más alta para el equipo. La principal desventaja de Flutter es aprender Dart (puntaje 3 en ese criterio).

2.4. El modelo de layout de Flutter

Flutter organiza la interfaz componiendo widgets; los widgets de layout son widgets invisibles que ordenan a otros [4]. Toda la lógica se resume en una regla de la documentación [5]:

«Las restricciones bajan. Los tamaños suben. El padre fija la posición.»

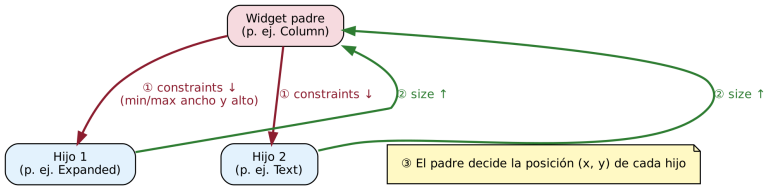


Figura 2: Protocolo de layout de Flutter: el padre pasa restricciones (mínimo y máximo de ancho y alto) a cada hijo, cada hijo devuelve un tamaño dentro de esos límites y el padre decide dónde colocarlo. Elaboración propia a partir de [5].

Esto ocurre en una sola pasada por el árbol, lo que hace el layout rápido, pero implica que un widget *no puede elegir libremente su tamaño ni su posición* [6, 5]. Las restricciones pueden

ser *estrictas* (mínimo = máximo, un solo tamaño posible, como la pantalla para el widget raíz) o *flexibles* (mínimo 0, como las que `Center` da a su hijo).

Widget	Comportamiento según la documentación [5]
`Container`	Sin tamaño ni hijo, intenta ser lo más grande posible; con hijo y sin tamaño, mide lo que su hijo (más el padding); con `width`/`height`, intenta respetarlos *solo dentro* de lo que permita su padre.
`Row` / `Column`	Con eje principal acotado, intentan ser lo más grandes posible; con eje principal sin límite (dentro de un scroll u otro flex) se ajustan a sus hijos y estos *no pueden* usar `Expanded`. El eje transversal nunca puede ser ilimitado. No limitan a sus hijos, así que pueden desbordar.
`Expanded` / `Flexible`	Dentro de `Row`/`Column` reparten el espacio sobrante según `flex`. `Expanded` obliga al hijo a ocupar su parte; `Flexible` deja que sea menor. Ambos ignoran el tamaño intrínseco del hijo.
`Align` / `Center`	Relajan las restricciones (el hijo puede ser más pequeño) y lo posicionan según `alignment`.
`ConstrainedBox`	Añade restricciones *adicionales* a las del padre; no puede saltárselas.
`UnconstrainedBox`	No impone restricciones a su hijo; si el hijo es mayor que la caja, aparece la advertencia de desborde.

Tabla 4: Comportamiento de los widgets de layout más usados.

2.5. Catálogo de widgets de layout usados en MatrLab

Los widgets de layout se clasifican en los que reciben un solo hijo (`child`) y los que reciben varios (`children`) [4]. La tabla 5 lista los que se usaron y dónde. Los marcados con † no están descritos en la página consultada y su descripción es conocimiento general del framework.

Widget	Tipo	Qué hace	Uso en MatrLab
`Scaffold`	1 hijo (ranuras)	Estructura base: `appBar`, `body`, barra inferior	Cada pantalla; el `ShellScreen`
`Column`	varios	Ordena en vertical	Formularios, tarjetas, hoja de confirmación
`Row`	varios	Ordena en horizontal	Fila de resúmenes, título + estado, botones
`Expanded`	1 hijo (en Row/Column)	Ocupa el espacio sobrante	Tres `StatTile` iguales; textos largos con `ellipsis`
`Spacer`†	sin hijo	Hueco flexible entre widgets de una fila	Separar «N grupos» del chip de estado
`Stack` / `Positioned`	varios	Superpone; `Positioned` coloca en un borde	Cabecera del curso (icono decorativo sobre degradado)
`Padding`, `SizedBox`	1 hijo	Margen interno; tamaño fijo o separación	Márgenes de 16 dp, separaciones de 4 a 24 dp, botón de 48 dp
`Align`, `Center`	1 hijo	Alinea o centra	Contenido centrado en login y confirmación
`ConstrainedBox`	1 hijo	Limita el ancho máximo	Login (420 dp), contenido en tablets (720 dp)
`Container`	1 hijo	Decoración, tamaño, color	Iconos circulares, chips, avisos, tarjetas de estado
`ClipRRect`†	1 hijo	Recorta con bordes redondeados	Barras de cupos, cabecera con degradado
`Card`	1 hijo	Caja Material con esquinas y sombra	Cursos, grupos, notificaciones
`ListView`	varios, con scroll	Lista que se desplaza	Detalle de curso y de grupo, horario, alertas, panel
`GridView`	varios, con scroll	Cuadrícula	Cursos (`.builder`, columnas según ancho); panel (`.count`)
`Table`	varios	Filas y columnas alineadas	Datos del grupo; horario semanal
`Wrap`†	varios	Salta de línea al quedarse sin espacio	Filtros y lista de espera
`IndexedStack`†	varios	Muestra uno y conserva los demás	Pestañas del `ShellScreen`
`SafeArea`†	1 hijo	Evita muescas y barras del sistema	Login, confirmación, botón inferior
`LayoutBuilder`	1 hijo (builder)	Entrega las restricciones del padre	Elegir barra inferior o riel lateral

Tabla 5: Widgets de organización y layout usados en la aplicación.

2.6. Equivalencias con React Native y .NET MAUI

Como el laboratorio pide investigar los componentes propios del framework, la tabla 6 traduce cada necesidad de layout a lo que ofrecen los otros dos. Las celdas con * son conocimiento general que las páginas consultadas no detallan.

Necesidad	Flutter	React Native	.NET MAUI
Apilar en vertical	`Column` [4]	`View`; `flexDirection` es `column` por defecto [10]	`VerticalStackLayout` [13]
Apilar en horizontal	`Row` [4]	`View` con `flexDirection: 'row'` [10]	`HorizontalStackLayout` [13]
Repartir espacio sobrante	`Expanded`/`Flexible` con `flex` [4]	Propiedad `flex: n` [10]	`Grid` con filas/columnas proporcionales [13]
Alinear hijos	`mainAxisAlignment`, `crossAxisAlignment` [4]	`justifyContent`, `alignItems` [10]	Alineación en cada control (`HorizontalOptions`*)
Superponer	`Stack` + `Positioned` [4, 9]	`position: 'absolute'` [10]	`AbsoluteLayout` [13]
Saltar de línea	`Wrap`†	`flexWrap: 'wrap'` [10]	`FlexLayout` (admite *wrap*) [13]
Separar elementos	`SizedBox`, `Padding`	`gap`, `rowGap`, `columnGap` [10]	Propiedades de espaciado y margen*
Lista con scroll eficiente	`ListView.builder` [9]	`FlatList` [9]	Control de colecciones (`CollectionView`*) [12]
Cuadrícula	`GridView` [4]	Filas con `flexWrap` y ancho en % [10]	`Grid` [13]
Estilos reutilizables	`ThemeData`, `TextStyle` [9]	`StyleSheet.create` [9]	Estilos y recursos XAML*

Tabla 6: Equivalencias de layout entre los tres frameworks.

Una diferencia importante: en React Native, `flexDirection` es `column` por defecto, `alignContent` es `flex-start` y `flexShrink` es 0, a diferencia de CSS [10]. En Flutter no existe un «contenedor flex» genérico: se elige `Row` o `Column` [9]. Y la documentación de MAUI advierte que no se debe imitar un `Grid` anidando pilas horizontales, ni usar `Grid` para datos tabulares [13].

2.7. Diseño adaptable

Flutter recomienda un proceso de tres pasos: *abstraer* los widgets que cambian, *medir* el espacio y *ramificar* según el tamaño [7]. Para medir hay dos herramientas: `MediaQuery.sizeOf` mide la ventana completa y es la indicada para elegir la navegación; `LayoutBuilder` mide el espacio que el padre le da a un widget concreto [7]. Los puntos de corte deben depender del tamaño de la ventana y no del tipo de dispositivo; la guía de Material usa una barra de navegación inferior bajo 600 dp y un riel lateral desde 600 dp [7].

En MatrLab se aplican ambas ideas: el `ShellScreen` usa `LayoutBuilder` con el corte de 600 dp (barra inferior *vs.* `NavigationRail`), y la lista de cursos usa `SliverGridDelegateWithMaxCrossAxisExtent` para pasar de una columna en el teléfono a dos o más en una tableta, sin escribir un corte explícito.

2.8. Componentes de Material usados

Además de los widgets de layout, la interfaz se construye con componentes del catálogo Material de Flutter [8]. Los marcados con † no figuran en la página consultada.

Componente	Qué es [8]	Dónde se usa
`AppBar`	Contenedor de contenido y acciones en la parte superior	Título de cada pestaña; cerrar sesión
`NavigationBar`	Cambia entre destinos principales	Pestañas en teléfono
`NavigationRail`†	Versión lateral de la barra de navegación	Pestañas en tableta
`Card`, `ListTile`	Caja con sombra; fila de altura fija con texto e iconos	Cursos, grupos, matrículas, notificaciones
`FilterChip`, `Chip`	Bloques pequeños para filtrar o mostrar una opción	Filtros de cursos; lista de espera
`TextField`/`TextFormField`	Cuadro de entrada de texto	Código y contraseña con validación
`FilledButton`†, `OutlinedButton`†, `TextButton`	Botones de énfasis alto, medio y bajo	Acciones principales y secundarias
`SegmentedButton`†	Selección entre pocas opciones excluyentes	Rol: estudiante o administrador
`SearchBar`†	Campo de búsqueda	Búsqueda de cursos
`Badge`†	Contador o punto sobre un icono	Alertas sin leer
`LinearProgressIndicator`†	Barra de progreso	Ocupación de cupos
`AlertDialog`, `SnackBar`	Diálogo de decisión; mensaje breve inferior	Confirmar cancelación; avisos de error
`showModalBottomSheet`†	Hoja que sube desde el borde inferior	Confirmar matrícula

Tabla 7: Componentes Material empleados.

2.9. Errores típicos de layout y cómo se evitaron

La documentación indica que el desborde se muestra con un patrón de rayas amarillas y negras, que un `Card` por defecto mide 0×0 y que ciertos widgets lanzan excepciones cuando reciben restricciones infinitas [4, 5]. La tabla 8 muestra los casos previstos en MatrLab.

Síntoma	Causa	Solución aplicada
Franja amarilla de desborde en una fila	Un `Row` no limita a sus hijos; el texto largo no cabe [5]	Texto dentro de `Expanded` con `maxLines` y `TextOverflow.ellipsis` (`_CursoCard`)
Excepción por altura ilimitada	`GridView` dentro de `ListView` recibe altura sin límite [5]	`GridView.count` con `shrinkWrap: true` y `NeverScrollableScrollPhysics` (panel)
`Expanded` dentro de un scroll	El eje principal es ilimitado y no hay espacio que repartir [5]	Los `Expanded` solo se usan en filas, y las columnas con scroll usan tamaños propios
El contenido se estira en pantallas anchas	El `ListView` ocupa todo el ancho	`ContenidoCentrado`: `Align` + `ConstrainedBox(maxWidth)`
El botón inferior queda bajo la barra del sistema	No se respeta el área segura	`SafeArea` en `bottomNavigationBar`
`Card` invisible	Sin tamaño propio se reduce a 0×0 [4]	Se coloca en contenedores con tamaño (`mainAxisExtent` de la cuadrícula, `ListView`)

Tabla 8: Problemas comunes de layout y su prevención en MatrLab.

3. Descripción general del programa

3.1. Contexto y problema

En el Laboratorio 1 se planteó una aplicación de apoyo a la matrícula de estudiantes en los laboratorios de la Escuela Profesional de Ingeniería de Sistemas, y se describieron las dificultades vividas por el equipo en procesos de matrícula similares: cupos que se agotan en segundos, falta de información de disponibilidad en tiempo real y saturación del sistema por alta concurrencia el día de matrícula [2]. En el Laboratorio 2 se construyó un prototipo con el modelo de datos Cursos, Grupos de Laboratorio, Usuarios y Matrículas [3]. Este laboratorio parte de ese mismo problema.

3.2. Árbol de problemas

El árbol de la figura 3 organiza el problema central, sus causas y sus efectos. Las causas C1 a C3 son las dificultades documentadas en el Laboratorio 1 [2]; las causas C4 a C6 y los efectos E1 a E4 se plantearon en este trabajo a partir de las funcionalidades que ya figuraban en la descripción de MatrLab (validación de horarios, notificaciones, panel administrativo y reportes de ocupación) y son **hipótesis del autor**, no resultados de una encuesta.

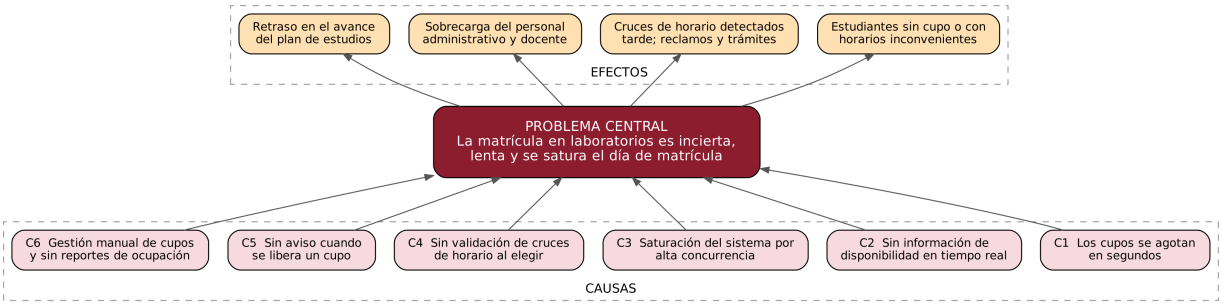


Figura 3: Árbol de problemas de MatrLab: seis causas (abajo), problema central (centro) y cuatro efectos (arriba).

3.3. De las causas a las funcionalidades

Cada causa se convierte en una funcionalidad concreta (requisito funcional, RF) y cada RF en una pantalla (figura 4 y tabla 9). Así el diseño de las interfaces queda justificado por el árbol y no por preferencia estética.

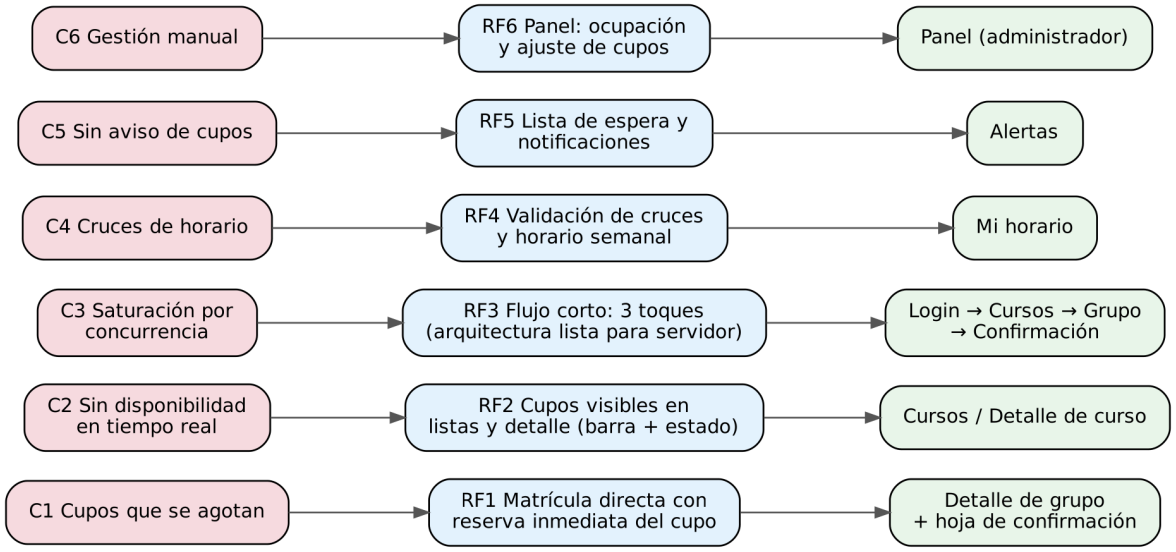


Figura 4: Trazabilidad causa → funcionalidad → pantalla.

Causa	RF	Funcionalidad	Pantalla
C1	RF1	Matrícula directa: al confirmar, el cupo se descuenta de inmediato y se genera la matrícula con estado «Confirmada». Un grupo lleno rechaza la solicitud.	Detalle de grupo, hoja de confirmación, confirmación
C2	RF2	La ocupación se muestra como barra, fracción y etiqueta con icono (Disponible / Casi lleno / Lleno) en la lista de cursos y en cada grupo.	Cursos, detalle de curso
C3	RF3	Flujo mínimo: curso → grupo → confirmar. La lógica está aislada en `MatrLabState`, de modo que pueda moverse a un servidor sin tocar las pantallas.	Todo el flujo
C4	RF4	Se impide matricularse en un grupo cuyo horario se cruce con otro ya matriculado, o en dos grupos del mismo curso; el horario semanal muestra las franjas ocupadas.	Detalle de grupo, mi horario
C5	RF5	Si el grupo está lleno, el estudiante se une a una lista de espera; cuando el administrador agrega cupos, se genera la notificación «Se liberó un cupo».	Detalle de grupo, alertas
C6	RF6	Panel con cupos totales y ocupados, ocupación global, ranking de grupos más saturados y botones para agregar o quitar cupos.	Panel (administrador)
—	RF7	Inicio de sesión con código de estudiante (8 dígitos) o usuario administrador, con validación de campos.	Login

Tabla 9: Requisitos funcionales derivados del árbol de problemas.

3.4. Requisitos no funcionales

Requisito	Cómo se atiende
Diseño adaptable	Navegación inferior bajo 600 dp y riel lateral desde 600 dp; cuadrícula de cursos con ancho máximo por tarjeta; contenido centrado con ancho máximo.
Accesibilidad básica	El estado de ocupación nunca depende solo del color: lleva icono y texto. Las barras de cupos tienen etiqueta semántica (`Semantics`). Los botones de icono tienen `tooltip`.
Consistencia visual	Un solo tema Material 3 generado desde un color semilla; widgets reutilizables (`CupoBar`, `EstadoChip`, `StatTile`, `Aviso`).
Mantenibilidad	Estado separado de la interfaz, una pantalla por archivo, sin paquetes externos (solo el SDK de Flutter).
Verificabilidad	Pruebas unitarias de las reglas de negocio, una prueba de interfaz del flujo completo y un conjunto de pruebas que generan las capturas.

Tabla 10: Requisitos no funcionales.

3.5. Datos de ejemplo

Curso	Grupo	Horario	Aula	Cupos
Plataformas Emergentes (IS-742)	A	Mar 14:00–16:00	Lab. Redes 302	17/20
	B	Jue 10:00–12:00	Lab. Redes 302	9/20
Seguridad Informática (IS-735)	A	Lun 08:00–10:00	Lab. Cómputo 201	15/18
	B	Mié 16:00–18:00	Lab. Cómputo 201	6/18
Base de Datos II (IS-624)	A	Mar 08:00–10:00	Lab. Cómputo 105	12/25
	B	Vie 14:00–16:00	Lab. Cómputo 105	25/25 (lleno)
Redes y Comunicación de Datos (IS-533)*	A	Mar 14:00–16:00	Lab. Redes 301	10/22
	B	Jue 14:00–16:00	Lab. Redes 301	21/22
Ingeniería de Software (IS-641)*	A	Mié 08:00–10:00	Lab. Cómputo 202	5/24

Tabla 11: Cursos y grupos de demostración (ocupados/total). Los de Plataformas Emergentes, Seguridad Informática y Base de Datos II provienen del Laboratorio 2 [3]; * son ficticios. Los casos están elegidos a propósito: PE-A y RC-A se cruzan; BD2-B está lleno; PE-A y RC-B están casi llenos.

4. Diseño de interfaces

4.1. Sistema de diseño

Antes de dibujar pantallas se fijaron las reglas visuales para que todas se vean como una sola aplicación (tabla 12).

Elemento	Decisión
Tema	Material 3 con `ColorScheme.fromSeed` y color semilla guinda (`#8C1D2F`), cercano a los colores de la universidad; los demás tonos salen del esquema.
Estados de ocupación	Disponible (verde), casi lleno (ámbar, desde 85 %) y lleno (rojo, sin cupos). Siempre con icono y texto, no solo color.
Espaciado	Múltiplos de 4 dp; márgenes de pantalla de 16 dp; separación entre tarjetas de 8 a 12 dp.
Formas	Esquinas redondeadas de 12 a 20 dp en tarjetas y avisos; botones del tema.
Jerarquía de texto	Estilos del tema (`titleLarge`, `titleMedium`, `bodyMedium`, `bodySmall`); títulos en negrita.
Objetivos táctiles	Botones principales de 48 dp de alto.
Componentes propios	`CupoBar`, `EstadoChip`, `StatTile`, `Aviso`, `EstadoVacio`, `ContenidoCentrado`.

Tabla 12: Sistema de diseño de MatrLab.

4.2. Mapa de navegación

La aplicación tiene un solo nivel de pestañas (dentro de `ShellScreen`) y un flujo de pantallas apiladas para matricularse (figura 5). El administrador ve las pestañas Cursos y Panel; el estudiante, Cursos, Mi horario y Alertas.

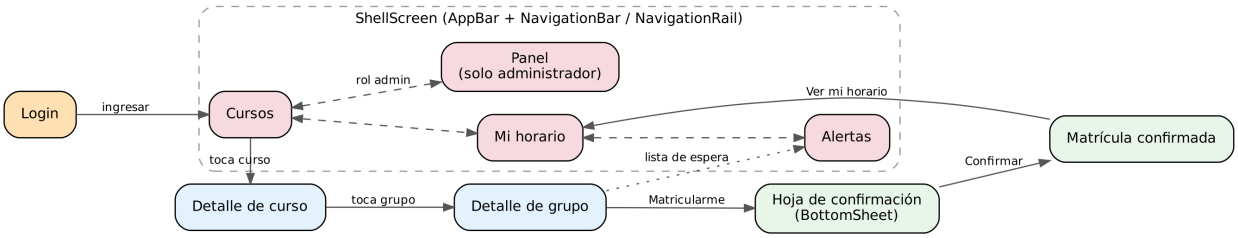


Figura 5: Mapa de navegación de MatrLab.

4.3. Mockups

Los mockups se dibujaron antes de programar, a escala de teléfono (360×740 dp), como wireframes de mediana fidelidad. Cada número rojo identifica un elemento y la tabla contigua indica **con qué widget de Flutter se construye**, que es lo que pide la guía para explicar el mockup.

Pantalla de inicio de sesión

Estructura: `Scaffold` → `SafeArea` → `Center` → `SingleChildScrollView` → `ConstrainedBox(maxWidth: 420)` → `Form` → `Column(stretch)`. El `ConstrainedBox` evita que el formulario se estire en pantallas anchas y el scroll evita desbordes cuando aparece el teclado.



N.°	Elemento	Widget
1	Logo circular	`Container` (`BoxDecoration` circular) con `Icon`, dentro de `Center`
2	Selector de rol	`SegmentedButton<Rol>`
3	Código y contraseña	`TextFormField` ×2 con `validator`; el segundo con `IconButton` para mostrar u ocultar
4	Botón Ingresar	`FilledButton`
5	Nota de demostración	`Text` con estilo `bodySmall`

Figura 6: Mockup: Pantalla de inicio de sesión.

Pestaña Cursos (inicio)

Estructura: `Column` con `Padding`+`SearchBar`, `Row` de tres `Expanded`, `Wrap` de filtros y un `Expanded` que contiene el `GridView.builder` de tarjetas (no numerado). Cada tarjeta es `Card` → `InkWell` → `Padding` → `Row`[icono, `Expanded`(`Column`)].



N.°	Elemento	Widget
1	Título y cierre de sesión	`AppBar` con `actions` (`Text` + `IconButton`)
2	Búsqueda	`SearchBar`
3	Tres resúmenes	`Row` con 3 `Expanded(StatTile)`
4	Filtros	`Wrap` de `FilterChip`
5	Navegación	`NavigationBar` (con `Badge` en Alertas)

Figura 7: Mockup: Pestaña Cursos (inicio).

Detalle de un curso

Estructura: `ListView` con `padding` de 16 dp; la cabecera es `ClipRRect` → `Container`(degradado) → `Stack`[`Positioned`(icono), `Padding`(`Column`)]. Los grupos se generan con un `for` dentro de `children`.



N.°	Elemento	Widget
1	Barra con flecha de regreso	`AppBar` (la flecha la agrega `Navigator`)
2	Cabecera con degradado	`Stack` + `Positioned`: el icono decorativo se superpone al texto
3	Descripción	`Text`
4	Tarjeta de grupo	`Card` → `InkWell` → `Column`; título y estado en una `Row`
5	Barra de cupos	`CupoBar`: `Column`[`Row`, `ClipRRect`(`LinearProgressIndicator`)]

Figura 8: Mockup: Detalle de un curso.

Detalle de un grupo

Estructura: `Scaffold` con `ListView` en el cuerpo y el botón de acción fijo en `bottomNavigationBar`, para que siempre esté al alcance del pulgar. El botón cambia según el caso: *Matricularme*, *Cancelar matrícula*, *Unirme a la lista de espera* o deshabilitado (cruce de horario, curso ya matriculado).

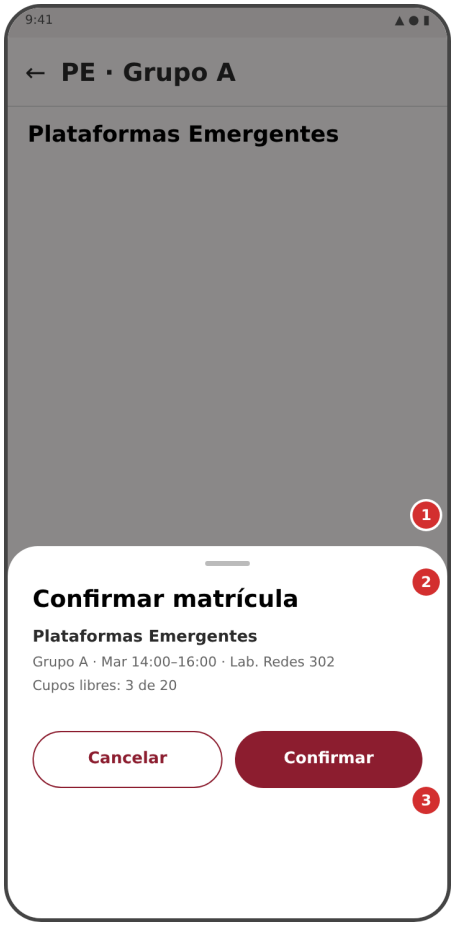


N.°	Elemento	Widget
1	Barra superior	`AppBar`
2	Título y estado	`Row[Expanded(Text), EstadoChip]`
6	Datos del grupo	`Table` con `FixedColumnWidth(124)` y `FlexColumnWidth`; una `TableRow` por dato
3	Cupos	`Card` → `Padding` → `CupoBar(grande: true)`
4	Aviso contextual	`Aviso: Container` → `Row[Icon, Expanded(Text)]`
5	Botón de acción	`SafeArea` → `Padding` → `SizedBox(48 dp)` → `FilledButton`

Figura 9: Mockup: Detalle de un grupo.

Hoja de confirmación de matrícula

Se abre con `showModalBottomSheet` sobre el detalle del grupo. Resume la elección antes de reservar el cupo, de modo que una pulsación accidental no matricule.



N.°	Elemento	Widget
1	Fondo atenuado	Barrera modal del `BottomSheet`
2	Hoja inferior	`SafeArea` → `Padding` → `Column(mainAxisSize: min)`, con `showDragHandle`
3	Cancelar / Confirmar	`Row[Expanded(OutlinedButton)`, `Expanded(FilledButton)]`

Figura 10: Mockup: Hoja de confirmación de matrícula.

Matrícula confirmada

Estructura: `Center` → `SingleChildScrollView` → `ConstrainedBox(440)` → `Column(min)`. Los botones llevan a la pestaña Mi horario o al inicio con `popUntil`.



N.°	Elemento	Widget
1	Icono de éxito	`Container` circular con `Icon` verde
2	Resumen	`Card` con `Column` de filas (`Row` + `Expanded` flex 2:3) separadas por `Divider`
3	Ver mi horario	`FilledButton`
4	Volver al inicio	`TextButton`

Figura 11: Mockup: Matrícula confirmada.

Pestaña Mi horario

El horario semanal es una `Table` de 6 columnas (la primera de 46 dp) y 6 filas (cabecera + cinco franjas de 2 h). Cada celda ocupada pinta la sigla del curso en un `Container` con esquinas redondeadas.

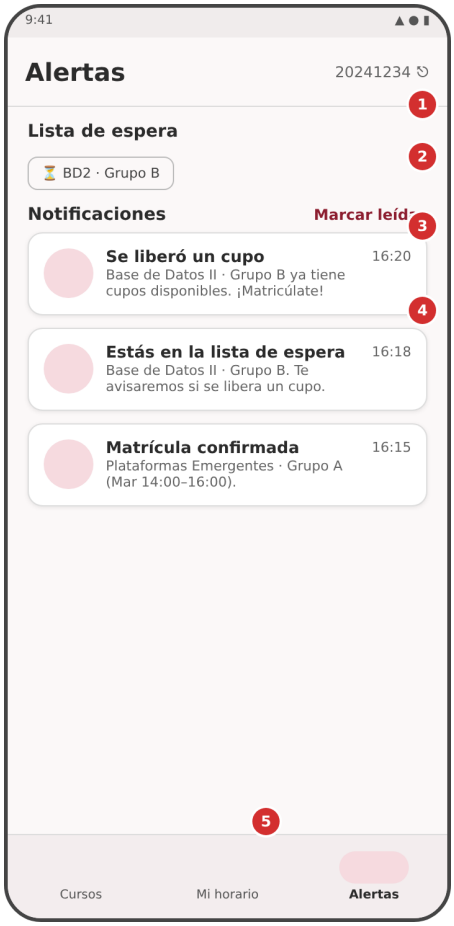


N.°	Elemento	Widget
1	Barra superior	`AppBar`
2	Resúmenes	`Row` con 2 `Expanded(StatTile)`
3	Horario semanal	`Table` con `TableBorder.all` y `FixedColumnWidth`
4	Mis matrículas	`Card` → `ListTile` (`leading` `CircleAvatar`, `trailing` `IconButton`)

Figura 12: Mockup: Pestaña Mi horario.

Pestaña Alertas

Si el estudiante tiene grupos en lista de espera, se muestran como `Chip`. El punto de «no leída» es un `Badge` sin etiqueta sobre el avatar.



N.°	Elemento	Widget
1	Barra superior	`AppBar`
2	Lista de espera	`Wrap` de `Chip`
3	Encabezado de notificaciones	`Row`[`Expanded`(`Text`), `TextButton`]
4	Notificación	`Card` → `ListTile` (`leading`: `Badge`+`CircleAvatar`; `trailing`: hora)
5	Navegación	`NavigationBar` con contador de no leídas

Figura 13: Mockup: Pestaña Alertas.

Pestaña Panel (administrador)

`ListView` con `GridView.count` (`shrinkWrap`, sin scroll propio) para los tres resúmenes, un ranking de ocupación con una fila `Row`[`SizedBox`, `Expanded`, `SizedBox`] por grupo y la gestión de cupos con `ListTile`.

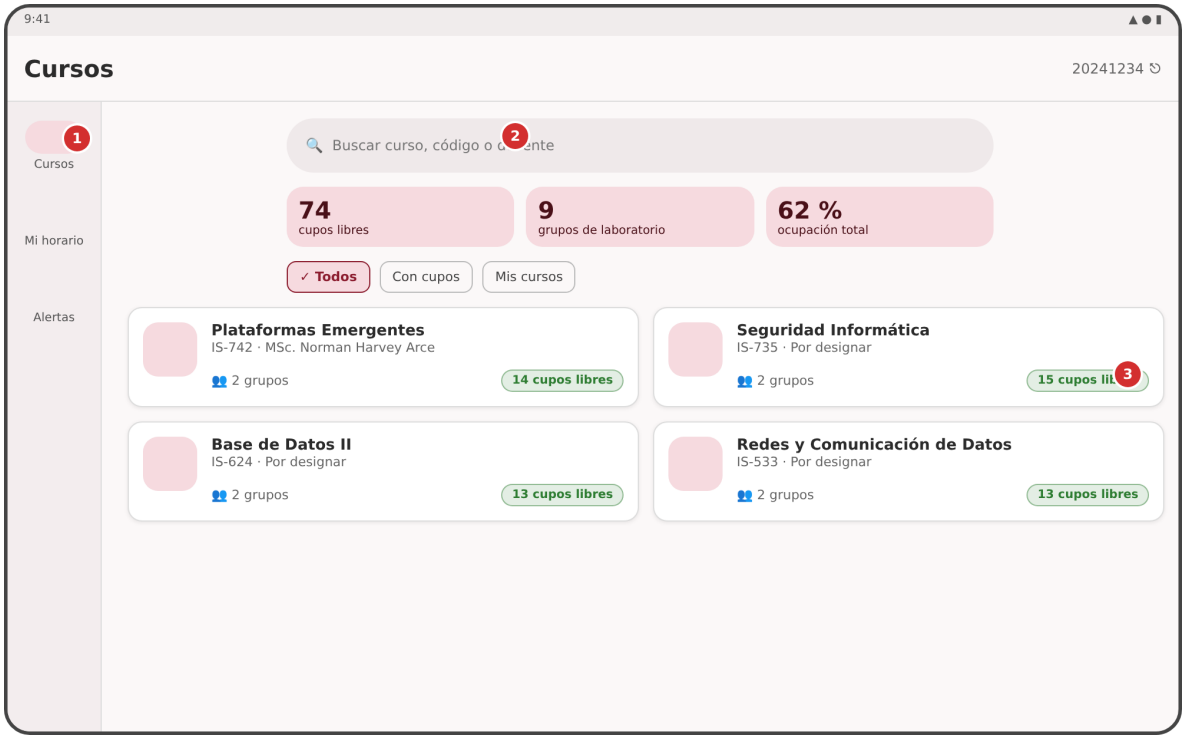


N.°	Elemento	Widget
1	Barra superior	`AppBar`
2	Resúmenes	`GridView.count(crossAxisCount: 3)`
3	Grupo con mayor presión	`Aviso` coloreado según el estado del grupo
4	Ocupación por grupo	`Row[SizedBox(74), Expanded(LinearProgressIndicator), SizedBox(56)]`
5	Gestión de cupos	`ListTile` con `trailing` = `SizedBox(128)` → `Row[IconButton, Text, IconButton]`

Figura 14: Mockup: Pestaña Panel (administrador).

Versión para tableta (1000 × 640 dp)

Con 600 dp o más, el `LayoutBuilder` del `ShellScreen` reemplaza la barra inferior por un `NavigationRail` dentro de una `Row`, y el cuerpo ocupa el resto con `Expanded`. La cuadrícula de cursos pasa a varias columnas porque cada tarjeta tiene un ancho máximo de 480 dp.



N.°	Elemento	Widget de Flutter
1	Riel de navegación	`Row`[`NavigationRail`, `VerticalDivider`, `Expanded`(cuerpo)]
2	Contenido con ancho máximo	`ContenidoCentrado(anchoMaximo: 1000)` (`Align` + `ConstrainedBox`)
3	Cuadrícula adaptable	`GridView.builder` con `SliverGridDelegateWithMaxCrossAxisExtent(480)`

Figura 15: Mockup: Versión para tableta (1000 × 640 dp).

5. Descripción del proceso de desarrollo

5.1. Entorno y creación del proyecto

El proyecto se crea con la herramienta de línea de comandos de Flutter. Se eligieron las plataformas Android y web (la web solo para revisar rápido el diseño); la aplicación no usa ningún paquete externo, únicamente el SDK.

```
flutter doctor                    # verifica SDK, Android Studio y emulador
flutter create --project-name matrlab --platforms android,web .
flutter pub get                   # descarga dependencias (solo el SDK)
flutter analyze                   # analisis estatico del codigo
flutter test                      # pruebas + generacion de capturas
flutter run                       # ejecuta en emulador o dispositivo
```

Listing 1: Comandos para preparar y verificar el proyecto.

El archivo `pubspec.yaml` declara solamente `flutter` (y `flutter_test` para las pruebas) y

activa `uses-material-design` para disponer de los iconos Material (archivo `pubspec.yaml` del repositorio).

5.2. Estructura del proyecto y arquitectura

```
lib/
  main.dart             # punto de entrada: runApp(MatrLabApp(...))
  app.dart              # MaterialApp, tema, decide Login o Shell
  models.dart           # Curso, GrupoLab, Horario, Matricula, Notificacion
  datos_demo.dart       # 5 cursos, 9 grupos, avisos iniciales
  estado.dart           # MatrLabState (reglas) + MatrLabScope (acceso)
  widgets/comunes.dart  # CupoBar, EstadoChip, StatTile, Aviso, ...
  screens/
    login_screen.dart shell_screen.dart cursos_screen.dart
    curso_detalle_screen.dart grupo_detalle_screen.dart
    confirmacion_screen.dart horario_screen.dart
    alertas_screen.dart panel_screen.dart
test/
  matrlab_state_test.dart # 17 pruebas de las reglas de negocio
  flujo_widget_test.dart  # flujo completo con widgets reales
  capturas_test.dart      # genera las 17 capturas del informe
```

Listing 2: Estructura de carpetas de `lib/` y `test/`.

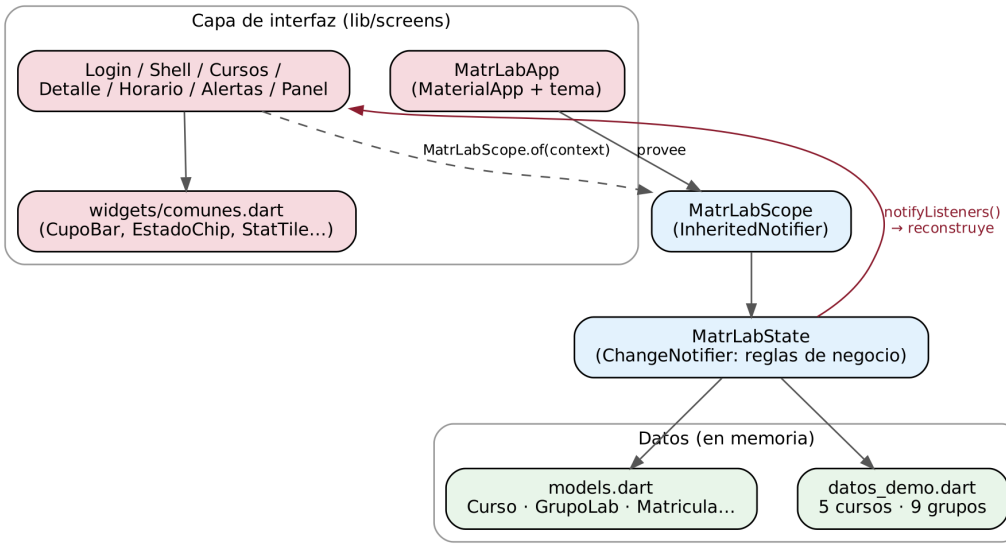


Figura 16: Arquitectura: las pantallas leen el estado a través de `MatrLabScope`; cuando el estado cambia llama a `notifyListeners()` y se reconstruyen las pantallas que dependen de él.

5.3. Paso 1: el estado y las reglas de negocio

Se separó la lógica de la interfaz. `MatrLabState` extiende `ChangeNotifier` y contiene los datos y las operaciones (`matricular`, `cancelarMatricula`, `unirseAListaEspera`, `ajustarCupos`,

`conflictoCon...`). `MatrLabScope` es un `InheritedNotifier`: un widget que coloca ese estado en lo alto del árbol y reconstruye automáticamente a todo widget que lo lea con `MatrLabScope.of(context)`. Así no se necesitó ningún paquete de gestión de estado. El método `matricular` aplica las reglas en orden: mismo curso, cruce de horario y cupo.

```dart
  ResultadoMatricula matricular(String grupoId) {
    final g = grupoPorId(grupoId);
    if (tieneCursoMatriculado(g.cursoCodigo)) {
      return ResultadoMatricula.yaMatriculado;
    }
    if (conflictoCon(g) != null) return ResultadoMatricula.cruceHorario;
    if (g.cuposLibres <= 0) return ResultadoMatricula.sinCupo;

    g.cuposOcupados++; // el cupo se reserva de inmediato
    matriculas.add(Matricula(grupoId: g.id, fecha: DateTime.now()));
    listaEspera.remove(g.id);
    _notificar(
      'Matrícula confirmada',
      '${cursoDeGrupo(g).nombre} · ${g.nombre} (${g.horario.texto}).',
      TipoNotificacion.matricula,
    );
    notifyListeners();
    return ResultadoMatricula.ok;
  }
```

Listing 3: Regla de matrícula: valida, reserva el cupo y notifica (`lib/estado.dart`).

```dart
/// Expone el [MatrLabState] a todo el árbol de widgets.
class MatrLabScope extends InheritedNotifier<MatrLabState> {
  const MatrLabScope({
    super.key,
    required MatrLabState estado,
    required super.child,
  }) : super(notifier: estado);

  /// Lee el estado y SUSCRIBE al widget: se reconstruye cuando cambia.
  static MatrLabState of(BuildContext context) {
    final scope = context.dependOnInheritedWidgetOfExactType<MatrLabScope>();
    assert(scope != null, 'Falta MatrLabScope encima en el árbol de widgets');
    return scope!.notifier!;
  }

  /// Lee el estado sin suscribirse (para usarlo dentro de callbacks).
  static MatrLabState leer(BuildContext context) {
    final scope = context.getInheritedWidgetOfExactType<MatrLabScope>();
    assert(scope != null, 'Falta MatrLabScope encima en el árbol de widgets');
    return scope!.notifier!;
  }
}
```

Listing 4: Acceso al estado desde cualquier widget (`lib/estado.dart`).

5.4. Paso 2: widgets reutilizables

Antes de las pantallas se construyeron los componentes que se repiten (sección 2.5). `CupoBar` es el ejemplo más importante: una `Column` con una `Row` (texto a la izquierda, fracción a la derecha mediante `Spacer`) y una barra de progreso recortada con `ClipRRect`. El color se calcula a partir del estado del grupo.

```dart
class CupoBar extends StatelessWidget {
  const CupoBar({super.key, required this.grupo, this.grande = false});

  final GrupoLab grupo;
  final bool grande;

  @override
  Widget build(BuildContext context) {
    final color = colorEstado(grupo.estado);
    final tema = Theme.of(context).textTheme;
    return Semantics(
      label:
          'Cupos ocupados ${grupo.cuposOcupados} de ${grupo.cuposTotales}',
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Text('Cupos ocupados', style: tema.bodySmall),
              const Spacer(),
              Text(
                '${grupo.cuposOcupados}/${grupo.cuposTotales}',
                style: (grande ? tema.titleMedium : tema.bodyMedium)
                    ?.copyWith(fontWeight: FontWeight.bold),
              ),
            ],
          ),
          const SizedBox(height: 6),
          ClipRRect(
            borderRadius: BorderRadius.circular(6),
            child: LinearProgressIndicator(
              value: grupo.ocupacion.clamp(0.0, 1.0).toDouble(),
              minHeight: grande ? 12 : 8,
              color: color,
              backgroundColor: color.withAlpha(40),
            ),
          ),
          if (grande) ...[
            const SizedBox(height: 6),
            Text(
              grupo.cuposLibres > 0
                  ? '${grupo.cuposLibres} cupos libres'
                  : 'Sin cupos libres',
              style: tema.bodySmall?.copyWith(color: color),
            ),
          ],
        ],
      ),
    );
  }
}
```

Listing 5: Componente CupoBar (`lib/widgets/comunes.dart`).

5.5. Paso 3: las pantallas

Cada pantalla se programó siguiendo su mockup (sección 4.3). A continuación se muestra, para cada una, la captura de la aplicación, el fragmento clave y el funcionamiento de sus widgets.

5.5.1. Inicio de sesión

Un `Form` con una `GlobalKey` agrupa los dos `TextFormField`; al pulsar *Ingresar* se llama a `validate()`, que ejecuta los `validator` y muestra los mensajes de error bajo cada campo. El `SegmentedButton<Rol>` cambia el rol, y con él la etiqueta y la regla del primer campo.

```
                  SegmentedButton<Rol>(
                    segments: const [
                      ButtonSegment<Rol>(
                        value: Rol.estudiante,
                        icon: Icon(Icons.school_outlined),
                        label: Text('Estudiante'),
                      ),
                      ButtonSegment<Rol>(
                        value: Rol.administrador,
                        icon: Icon(Icons.admin_panel_settings_outlined),
                        label: Text('Administrador'),
                      ),
                    ],
                    selected: {_rol},
                    onSelectionChanged: (s) => setState(() => _rol = s.first),
                  ),
                  const SizedBox(height: 20),
                  TextFormField(
                    controller: _codigo,
                    keyboardType: esEstudiante
                        ? TextInputType.number
                        : TextInputType.text,
                    textInputAction: TextInputAction.next,
                    decoration: InputDecoration(
                      labelText: esEstudiante
                          ? 'Código de estudiante'
                          : 'Usuario institucional',
                      prefixIcon: const Icon(Icons.badge_outlined),
                      border: const OutlineInputBorder(),
                    ),
                    validator: (v) => validarCodigo(v, _rol),
```

Listing 6: Selector de rol y campo de código (`lib/screens/login_screen.dart`).

5.5.2. Estructura principal y lista de cursos

`ShellScreen` es un `Scaffold` cuya barra de navegación depende del ancho disponible: un `LayoutBuilder` entrega las restricciones y, desde 600 dp, se muestra un `NavigationRail` dentro de una `Row`; si no, una `NavigationBar` inferior. Las tres pestañas viven en un `IndexedStack`, que conserva el estado de cada una (por ejemplo, lo escrito en la búsqueda) al cambiar de pestaña.

```
    return LayoutBuilder(
      builder: (context, restricciones) {
        final ancho = restricciones.maxWidth >= 600;
        return Scaffold(
          appBar: AppBar(
            title: Text(destinos[indice].titulo),
            actions: [
              Center(
                child: Padding(
                  padding: const EdgeInsets.only(right: 4),
                  child: Text(
                    e.codigoUsuario ?? '',
```



(a) Pantalla inicial: logo, selector de rol, campos y botón.



(b) Al pulsar Ingresar con los campos vacíos: mensajes de validación.

Figura 17: Inicio de sesión.

```
                  style: Theme.of(context).textTheme.labelMedium,
                ),
              ),
            ),
            IconButton(
              tooltip: 'Cerrar sesión',
              icon: const Icon(Icons.logout),
              onPressed: e.cerrarSesion,
            ),
          ],
        ),
        body: ancho
            ? Row(
                children: [
                  NavigationRail(
                    selectedIndex: indice,
                    onDestinationSelected: e.irAPestana,
                    labelType: NavigationRailLabelType.all,
                    destinations: [
                      for (final d in destinos)
                        NavigationRailDestination(
                          icon: _icono(e, d, false),
```

```
                            selectedIcon: _icono(e, d, true),
                            label: Text(d.titulo),
                          ),
                      ],
                    ),
                    const VerticalDivider(width: 1),
                    Expanded(child: cuerpo),
                  ],
                )
              : cuerpo,
```

Listing 7: Navegación adaptable (`lib/screens/shell_screen.dart`).

En `CursosScreen`, la cuadrícula se define con un `SliverGridDelegateWithMaxCrossAxisExtent`: cada tarjeta mide como máximo 480 dp de ancho y 128 dp de alto, y Flutter calcula cuántas columnas caben. En un teléfono resulta una sola columna; en una tableta, dos o más.

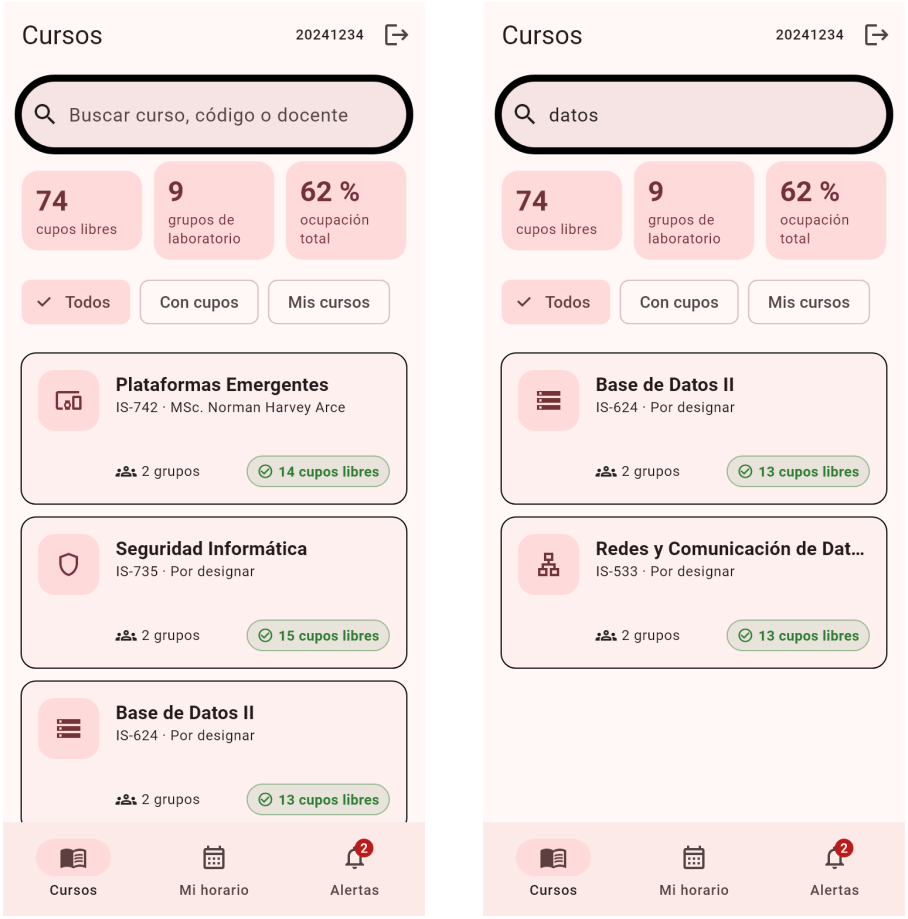
```
                : GridView.builder(
                    padding: const EdgeInsets.all(16),
                    gridDelegate:
                        const SliverGridDelegateWithMaxCrossAxisExtent(
                      maxCrossAxisExtent: 480,
                      mainAxisExtent: 128,
                      mainAxisSpacing: 12,
                      crossAxisSpacing: 12,
                    ),
                    itemCount: cursos.length,
                    itemBuilder: (context, i) => _CursoCard(curso: cursos[i]),
```

Listing 8: Cuadrícula adaptable de cursos (`lib/screens/cursos_screen.dart`).

5.5.3. Detalle de curso y de grupo

La cabecera del curso es un `Stack`: el primer hijo (un `Padding` con el texto) queda debajo y el `Positioned` del icono decorativo se coloca en la esquina inferior derecha, parcialmente fuera; el `ClipRRect` recorta lo que sobresale. En el detalle del grupo, el botón de acción se decide según el estado: matricularse, cancelar, lista de espera o botón deshabilitado con la razón.

```
            // Cabecera: Stack permite superponer el icono decorativo al texto.
            ClipRRect(
              borderRadius: BorderRadius.circular(20),
              child: Container(
                height: 150,
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    begin: Alignment.topLeft,
                    end: Alignment.bottomRight,
                    colors: [scheme.primary, scheme.secondary],
                  ),
                ),
                child: Stack(
                  children: [
                    Positioned(
                      right: -12,
                      bottom: -16,
                      child: Icon(
                        curso.icono,
                        size: 130,
```



(a) Pestaña Cursos: búsqueda, tres resúmenes, filtros y tarjetas.

(b) Al escribir «datos» solo queda Base de Datos II.

Figura 18: Lista de cursos con búsqueda.

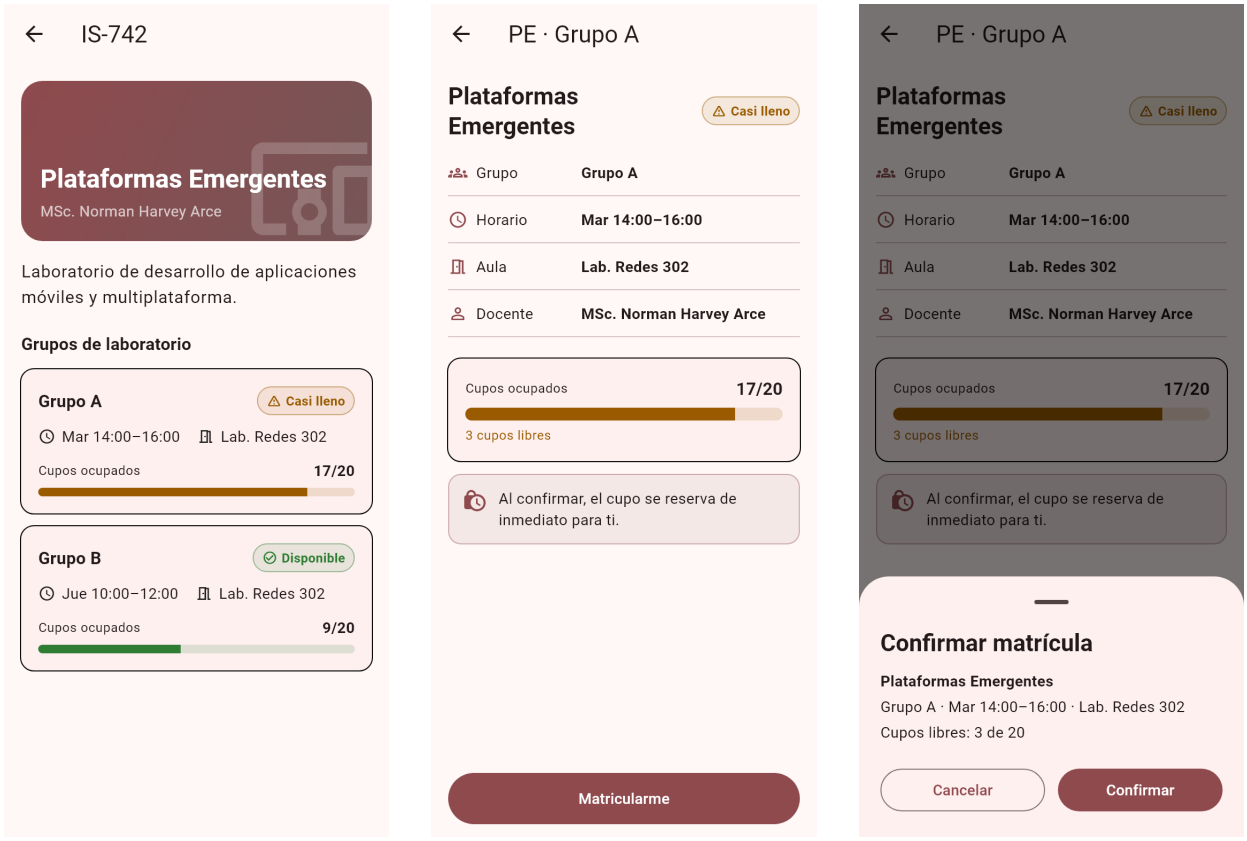
```
                    color: Colors.white.withAlpha(50),
                  ),
                ),
                Padding(
                  padding: const EdgeInsets.all(18),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.end,
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        curso.nombre,
                        style: tema.headlineSmall?.copyWith(
                          color: Colors.white,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      const SizedBox(height: 4),
                      Text(
                        curso.docente,
                        style: tema.bodyMedium
                            ?.copyWith(color: Colors.white70),
                      ),
```

```
                        ],
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
```

Listing 9: Cabecera con Stack y Positioned (`lib/screens/curso_detalle_screen.dart`).

```dart
Widget _accion(BuildContext context, MatrLabState e, GrupoLab g) {
  const clave = ValueKey('accion-grupo');
  if (e.esAdmin) {
    return const SizedBox.shrink();
  }
  if (e.estaMatriculado(g.id)) {
    return OutlinedButton.icon(
      key: clave,
      onPressed: () => _cancelar(context, e, g),
      icon: const Icon(Icons.event_busy),
      label: const Text('Cancelar matrícula'),
    );
  }
  if (e.tieneCursoMatriculado(g.cursoCodigo)) {
    return const FilledButton(
      key: clave,
      onPressed: null,
      child: Text('Ya tienes un grupo de este curso'),
    );
  }
  if (e.conflictoCon(g) != null) {
    return const FilledButton(
      key: clave,
      onPressed: null,
      child: Text('Cruce de horario'),
    );
  }
  if (g.cuposLibres <= 0) {
    if (e.enListaEspera(g.id)) {
      return const FilledButton(
        key: clave,
        onPressed: null,
        child: Text('Ya estás en la lista de espera'),
      );
    }
    return FilledButton.tonal(
      key: clave,
      onPressed: () => e.unirseAListaEspera(g.id),
      child: const Text('Unirme a la lista de espera'),
    );
  }
  return FilledButton(
    key: clave,
    onPressed: () => _matricular(context, e, g),
    child: const Text('Matricularme'),
  );
}
```

Listing 10: Botón de acción según el estado del grupo (`lib/screens/grupo_detalle_screen.dart`).



(a) Detalle del curso con sus dos grupos.

(b) Detalle del grupo: tabla de datos, cupos y botón fijo.

(c) Hoja de confirmación sobre el detalle.

Figura 19: Del curso al grupo y la confirmación de la matrícula.

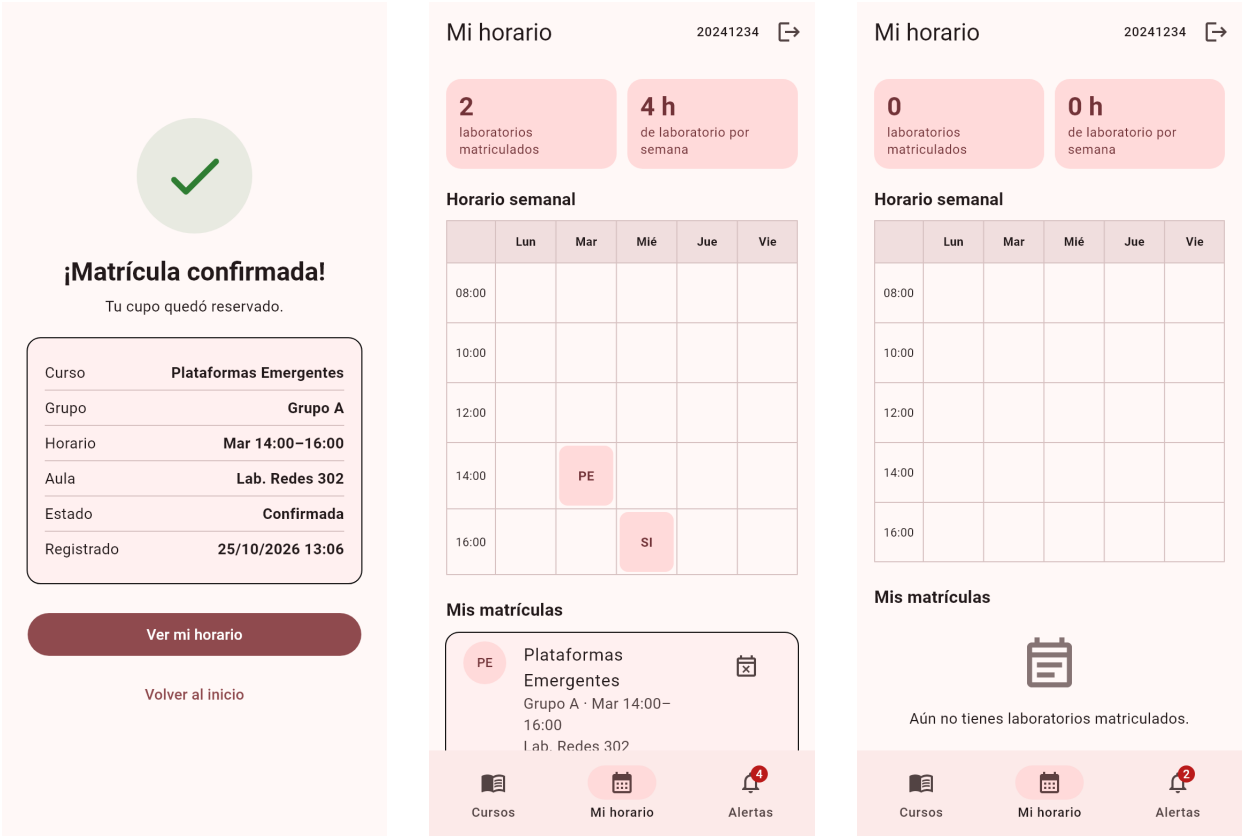
5.5.4. Confirmación y horario

Al confirmar, `matricular` reserva el cupo y la pantalla se reemplaza (`pushReplacement`) por la de éxito, de modo que el botón «atrás» no vuelva a la hoja. El horario semanal es una `Table`: la primera fila son los días y cada fila siguiente es una franja de dos horas; `_grupoEn` busca si alguna matrícula ocupa esa celda y, de ser así, se pinta la sigla del curso.

```
          Table(
            columnWidths: const {0: FixedColumnWidth(46)},
            defaultVerticalAlignment: TableCellVerticalAlignment.middle,
            border: TableBorder.all(color: scheme.outlineVariant),
            children: [
              TableRow(
                decoration: BoxDecoration(color: scheme.surfaceContainerHighest),
                children: [
                  _celdaTexto(''),
                  for (final d in _dias) _celdaTexto(d, negrita: true),
                ],
              ),
              for (final hora in _franjas)
                TableRow(
                  children: [
```

```
                    _celdaTexto('${hora.toString().padLeft(2, '0')}:00'),
                    for (var dia = 1; dia <= 5; dia++)
                      _celdaCurso(context, e, _grupoEn(e, dia, hora)),
                  ],
                ),
            ],
          ),
```

Listing 11: Horario semanal con Table (`lib/screens/horario_screen.dart`).



(a) Matrícula confirmada con el resumen.

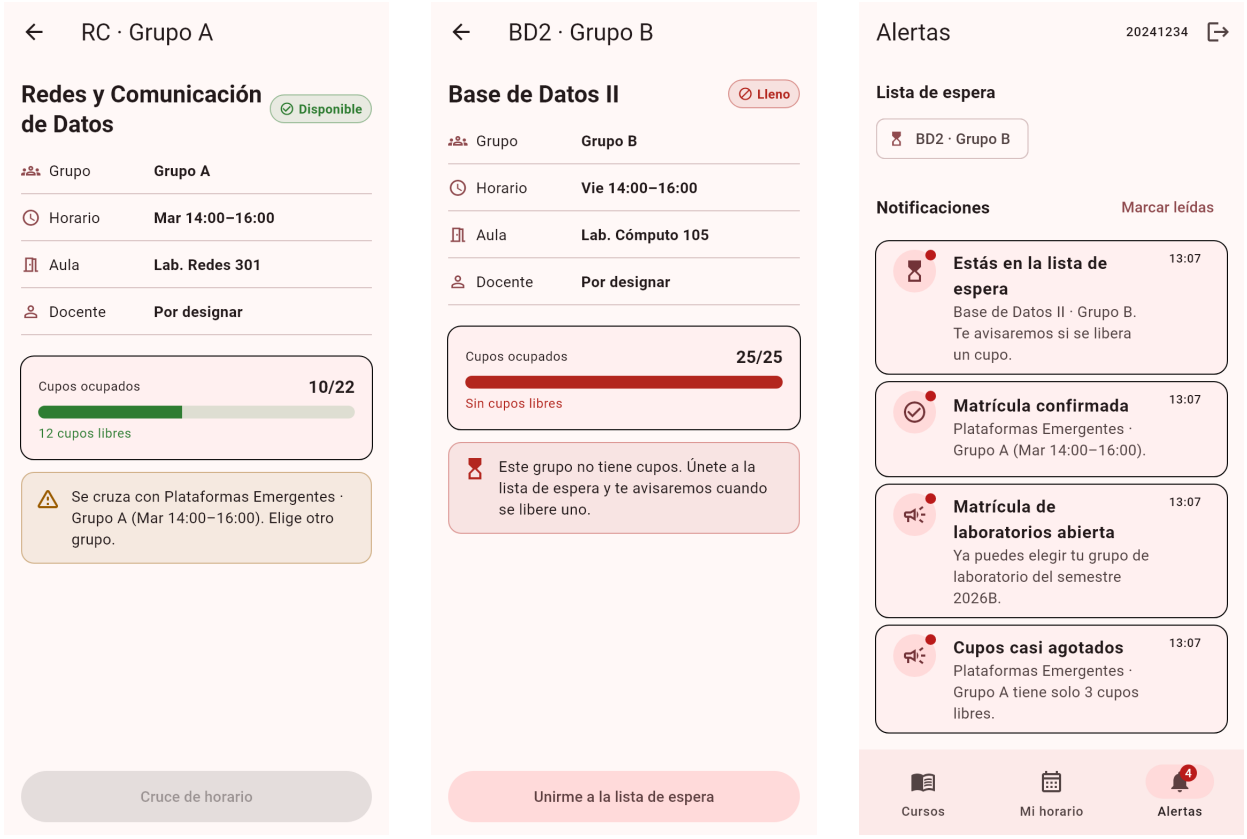
(b) Horario con PE (martes) y SI (miércoles).

(c) Estado vacío: sin matrículas, con acción.

Figura 20: Confirmación y horario semanal.

5.5.5. Casos especiales: cruce, grupo lleno y lista de espera

Las reglas de negocio se ven en la interfaz. Con PE grupo A matriculado, el grupo A de Redes (mismo horario) muestra un aviso ámbar y el botón queda deshabilitado. En BD2 grupo B, que está lleno, el botón pasa a «Unirme a la lista de espera»; al unirse, el grupo aparece como `Chip` en Alertas.



(a) Cruce con PE grupo A: botón deshabilitado y aviso.

(b) Grupo sin cupos: lista de espera.

(c) Alertas: lista de espera y notificaciones.

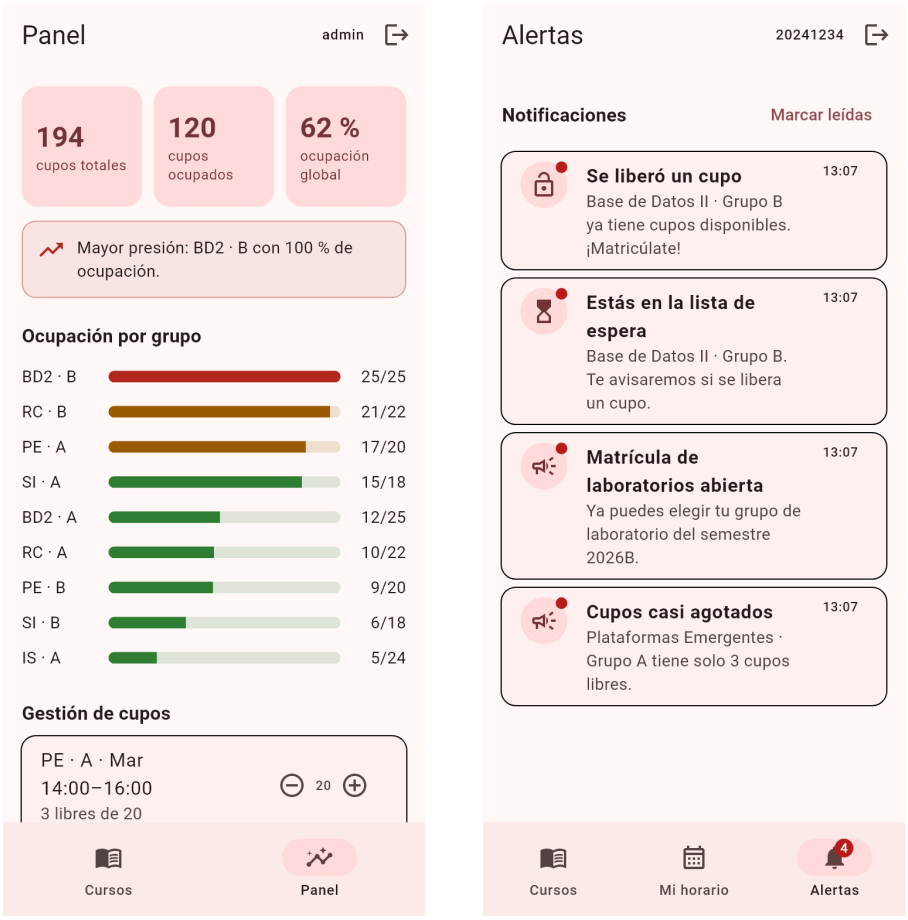
Figura 21: Casos que resuelven las causas C4 y C5 del árbol de problemas.

5.5.6. Panel del administrador

El panel usa un `GridView.count` dentro de un `ListView`. Como el `ListView` da altura ilimitada, el `GridView` necesita `shrinkWrap: true` y desactivar su propio scroll; sin eso Flutter lanzaría una excepción (tabla 8).

```
        GridView.count(
          crossAxisCount: 3,
          shrinkWrap: true,
          physics: const NeverScrollableScrollPhysics(),
          crossAxisSpacing: 10,
          mainAxisSpacing: 10,
          childAspectRatio: 1.0, // CAMBIO: antes 1.25; las tarjetas se desbordaban.
          children: [
            StatTile(valor: '${e.cuposTotales}', etiqueta: 'cupos totales'),
            StatTile(valor: '${e.cuposOcupados}', etiqueta: 'cupos ocupados'),
            StatTile(
              valor: '${(e.ocupacionGlobal * 100).round()} %',
              etiqueta: 'ocupación global',
            ),
          ],
        ),
```

Listing 12: Resúmenes del panel (`lib/screens/panel_screen.dart`).



(a) Panel: resúmenes, grupo más saturado, ranking y gestión de cupos.

(b) Tras agregar un cupo a BD2 grupo B, el estudiante recibe «Se liberó un cupo».

Figura 22: Causas C6 y C5: gestión de cupos y aviso al estudiante.

5.5.7. Diseño adaptable

La misma aplicación, sin cambiar código, en una ventana de 1000×640 dp: aparece el riel de navegación, el contenido se centra con ancho máximo y la lista de cursos pasa a dos columnas.

5.6. Del mockup a la aplicación

La figura 25 compara tres mockups con las pantallas implementadas. Las diferencias son de detalle (tipografía real, iconos reales en lugar de bloques); la estructura de cada pantalla se conserva.

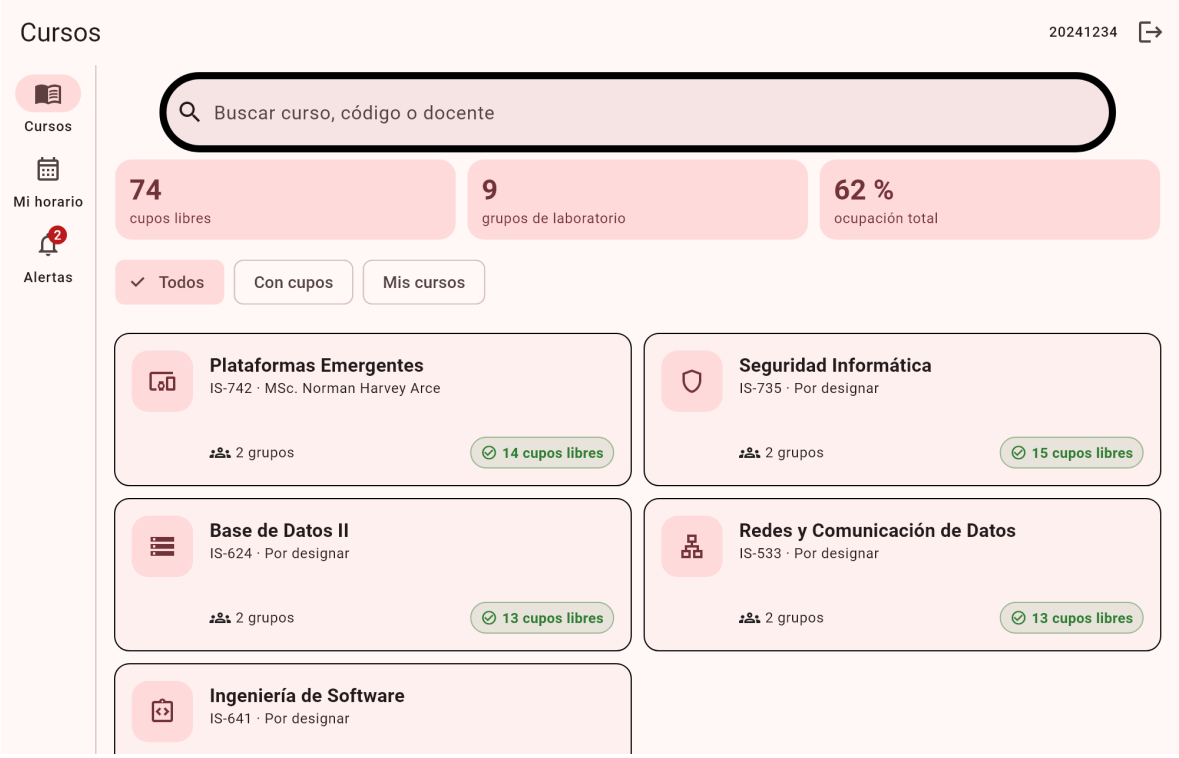
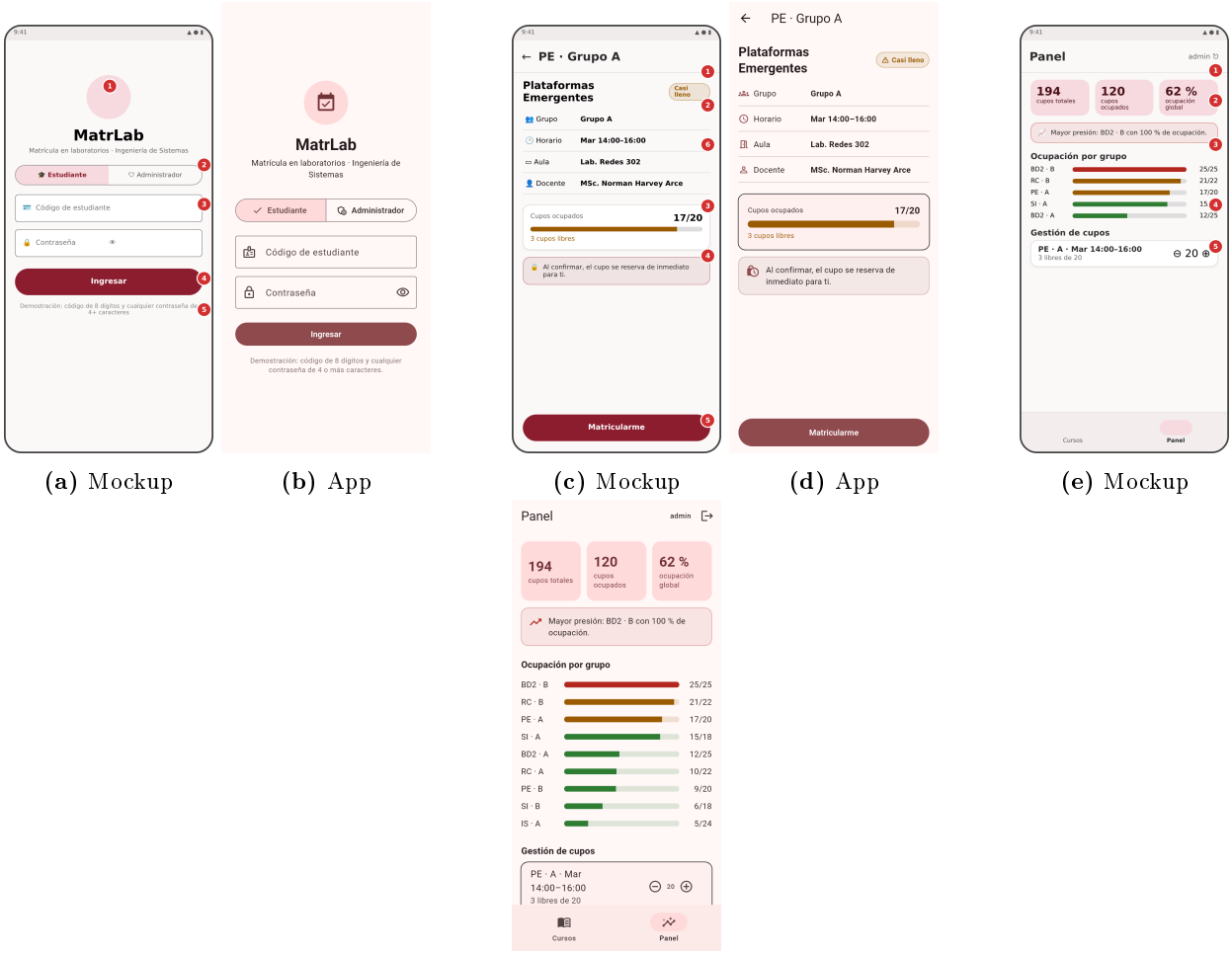


Figura 23: Cursos en tableta: `NavigationRail` y cuadrícula de dos columnas.



Figura 24: Horario en tableta: el contenido se limita a 720 dp y se centra.



(a) Mockup **(b)** App **(c)** Mockup **(d)** App **(e)** Mockup

(f) App

Figura 25: Mockup frente a implementación: login, detalle de grupo y panel.

5.7. Pruebas

Escribí cuatro conjuntos de pruebas (carpeta `test/` del repositorio):

- **Reglas de negocio** (`matrlab_state_test.dart`, 17 pruebas): validación de credenciales, matrícula con reserva de cupo, un solo grupo por curso, cruce de horario, grupo lleno, cancelación, lista de espera, notificación al agregar cupos, umbrales de estado, ranking y métricas, y solapamiento de horarios.
- **Flujo de interfaz** (`flujo_widget_test.dart`, 1 prueba): login con validación, curso, grupo, hoja de confirmación, pantalla de éxito y horario, con widgets reales.
- **Arranque** (`widget_test.dart`, 1 prueba): la aplicación abre en el inicio de sesión.
- **Capturas** (`capturas_test.dart`, 17 pruebas): llevan la app a un punto del flujo y guardan una imagen en `capturas/`.

Ejecuté el análisis estático y las pruebas en mi equipo; la salida final fue la siguiente (el análisis

no encontró problemas y las 36 pruebas pasaron).

Listing 13: Resultado de `flutter analyze` y `flutter test` (25/10/2026).

```
> flutter analyze
Analyzing matrlab-flutter...
No issues found! (ran in 2.2s)

> flutter test --reporter expanded
00:00 +0: ... capturas_test.dart: (setUpAll)
...
00:03 +35: ... capturas_test.dart: c17 tablet: horario
00:03 +36: ... capturas_test.dart: (tearDownAll)
00:03 +36: All tests passed!
```

Las primeras ejecuciones mostraron problemas que corregí antes de esta versión final: el archivo de plantilla `widget_test.dart` (creado por `flutter create`) hacía fallar el análisis; la fila de «N grupos» de las tarjetas de curso y las tarjetas de cifras del panel se desbordaban con la fuente real; y las pruebas dibujaban los iconos como cuadrados por no cargar la fuente de iconos. Los cambios están en la tabla 13.

5.8. Registro de cambios

Como pide la guía, los cambios sobre el trabajo se registran; en el código se marcan con comentarios `// CAMBIO:`.

Versión	Fecha	Cambio
1.0	24/10/2026	Primera versión: investigación, mockups, aplicación Flutter de 9 pantallas, pruebas e informe.
1.1	25/10/2026	Al ejecutar las pruebas en el equipo: la fila «N grupos» de las tarjetas de curso usa `Expanded` en lugar de `Spacer`; las tarjetas del panel pasan de proporción 1,25 a 1,0; las pruebas cargan Roboto y la fuente de iconos; se reemplaza la prueba de plantilla por una de arranque; se agregan las capturas reales.

Tabla 13: Registro de cambios del trabajo.

6. Apreciación de la herramienta

Se señalan dos características a favor y dos en contra de Flutter, cada una con la evidencia que dejó este proyecto.

6.1. Dos características a favor

1. **Layout declarativo y componible, con un widget por decisión.** Alinear, repartir espacio, superponer, recortar o hacer scroll son widgets con nombre propio, y la regla única «las restricciones bajan, los tamaños suben» [5] permite razonar cualquier pantalla de la misma forma. En MatrLab esto se tradujo en reutilización real: `StatTile` se usa 8 veces en 3 pantallas, `ContenidoCentrado` 6 veces y `CupoBar` aparece tanto en el detalle del curso

como en el del grupo, siempre con el mismo aspecto. Además, el catálogo de widgets de la sección 2.5 es pequeño (unos 18 widgets de layout) y cubrió las 9 pantallas.

2. **Un solo código que se adapta, sin paquetes externos.** La misma aplicación pasa de teléfono a tableta cambiando únicamente la navegación, con un `LayoutBuilder` y un punto de corte de 600 dp, y la cuadrícula de cursos se adapta sola con `SliverGridDelegateWithMaxCrossAxisExtent`. Todo el proyecto (2 262 líneas en `lib/`) se hizo solo con el SDK: Material 3, tema, navegación, formularios, tablas y estado (`ChangeNotifier` + `InheritedNotifier`) vienen incluidos. Además, el desarrollo en Flutter admite recarga en caliente con estado conservado [6].

6.2. Dos características en contra

1. **Anidamiento profundo de widgets.** Como todo es un widget y cada ajuste (margen, alineación, recorte) es otro widget envolvente, el código se hunde hacia la derecha. En `cursos_screen.dart` la tarjeta de curso llega a 18 niveles de sangría (`Card` → `InkWell` → `Padding` → `Row` → `Expanded` → `Column` → `Row`...), lo que obliga a extraer clases y funciones para mantener la legibilidad; la propia documentación recomienda separar partes de la interfaz en variables o funciones [4].

2. **Las reglas de restricciones generan errores en tiempo de ejecución que no son evidentes.** Un `Expanded` dentro de un contenedor con scroll, un `GridView` dentro de un `ListView` o un `Card` sin tamaño producen desbordes, excepciones o widgets invisibles [5, 4]. En el panel hubo que recordar `shrinkWrap: true` y `NeverScrollableScrollPhysics` para evitar el error (tabla 8). A esto se suma aprender Dart y su modelo de widgets, que para quien viene de JavaScript o C# es una barrera inicial (puntaje 3 en la tabla 3); y, a diferencia de React Native, Flutter no usa los controles del sistema sino los suyos [6, 11], de modo que la apariencia nativa depende del tema que se defina.

7. Conclusiones

- Partir de un **árbol de problemas** ordenó el diseño: cada causa tiene una funcionalidad y cada funcionalidad una pantalla (figura 4), de modo que no hay pantallas sin justificación ni causas sin respuesta, salvo la concurrencia masiva (C3), que solo puede resolverse con un servidor.
- Entre los tres frameworks analizados, **Flutter** resultó el más adecuado para este laboratorio por su modelo de layout explícito y consistente (matriz de la tabla 3); React Native queda como alternativa cercana si se prefiere JavaScript, y .NET MAUI si el equipo ya trabaja en el ecosistema .NET.
- El conocimiento clave de layout en Flutter cabe en una regla (restricciones abajo, tamaños arriba, posición decidida por el padre) y en un catálogo reducido de widgets; la mayoría de los errores se explican con esa regla.
- Se logró una aplicación completa sin paquetes externos, con lógica de negocio separada y probada, y con una interfaz que se adapta de teléfono a tableta.

- **Trabajo futuro:** conectar MatrLab a un servidor y una base de datos para resolver la concurrencia real (C3) y las notificaciones push; comparar el esfuerzo con el prototipo de Adalo del Laboratorio 2 [3]; y probar la aplicación en dispositivos Android reales.

Referencias

[1] Calienes Rodríguez, R. F. (2024). *Guía de laboratorio 4: Framework multiplataforma. Plataformas Emergentes (E)*. Universidad Nacional de San Agustín de Arequipa.

[2] Hilacondo Begazo, A. J. (2026). *Informe de Laboratorio 01: comparación de herramientas de desarrollo sin código (MatrLab)*. Plataformas Emergentes, UNSA.

[3] Hilacondo Begazo, A. J. (2026). *Informe de Laboratorio 02: desarrollo rápido de aplicaciones móviles: prototipo funcional en Adalo (MatrLab)*. Plataformas Emergentes, UNSA, 12 de septiembre de 2026.

[4] Flutter. (s. f.). *Layouts in Flutter*. https://docs.flutter.dev/ui/layout. Consultado el 24 de octubre de 2026.

[5] Flutter. (s. f.). *Understanding constraints*. https://docs.flutter.dev/ui/layout/constraints. Consultado el 24 de octubre de 2026.

[6] Flutter. (s. f.). *Architectural overview*. https://docs.flutter.dev/resources/architectural-overview. Consultado el 24 de octubre de 2026.

[7] Flutter. (s. f.). *General approach to adaptive apps*. https://docs.flutter.dev/ui/adaptive-responsive/general. Consultado el 24 de octubre de 2026.

[8] Flutter. (s. f.). *Material Components widgets*. https://docs.flutter.dev/ui/widgets/material. Consultado el 24 de octubre de 2026.

[9] Flutter. (s. f.). *Flutter for React Native developers*. https://docs.flutter.dev/get-started/flutter-for/react-native-devs. Consultado el 24 de octubre de 2026.

[10] Meta Platforms. (s. f.). *Layout with Flexbox*. React Native. https://reactnative.dev/docs/flexbox. Consultado el 24 de octubre de 2026.

[11] Meta Platforms. (s. f.). *Core Components and Native Components*. React Native. https://reactnative.dev/docs/intro-react-native-components. Consultado el 24 de octubre de 2026.

[12] Microsoft. (s. f.). *What is .NET MAUI?* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui. Consultado el 24 de octubre de 2026.

[13] Microsoft. (s. f.). *Layouts: .NET MAUI*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/user-interface/layouts/. Consultado el 24 de octubre de 2026.

A. Cómo reproducir el proyecto y las capturas

1. Instalar Flutter y Android Studio; comprobar con `flutter doctor`.
2. Abrir una terminal en la carpeta del proyecto y ejecutar `flutter create -project-name matrlab -platforms android,web .` (solo la primera vez; añade las carpetas de plataforma sin sobrescribir `lib/` ni `pubspec.yaml`).
3. `flutter pub get`, `flutter analyze` y `flutter test`. Esto ejecuta todas las pruebas y deja las 17 capturas en `capturas/`.
4. `flutter run` para abrir la aplicación en un emulador o en el navegador (`flutter run -d chrome`).